TECHNICAL INFRASTRUCTURE DEVELOPMENT REPORT

Private Cloud Cluster & AI Infrastructure

From Ngrok Exploration to a Full 4-Node Distributed System

Report Type	Technical Development Documentation
Project Name	Private Cloud / Homelab Cluster (4-Node Distributed System)
Project Phase	Planning, Architecture, Installation & Deployment
Technology Stack	Ubuntu Server, Docker, Nginx, Ollama, Spring Boot, DuckDNS, Cloudflare
Document Status	Complete — Full Session Summary
Classification	Internal / Academic Use

Prepared from a technical consultation session documenting the complete journey from initial server concept to live infrastructure deployment.

Table of Contents

1. Executive Summary 3
2. Background & Initial Problem Statement 3
3. Discovery: Why Ngrok is Not a Production Solution 4
4. Project Vision: Building a Mini Data Center 4
5. System Architecture Design 5
6. Technical Challenges & Problems Faced 6
7. Operating System Selection & Installation 7
8. Step-by-Step Installation Chronicle 8
9. AI Infrastructure: Ollama & Local LLM Deployment 10
10. Remote Access Setup: SSH & Cloudflare Tunnel 11
11. Academic Subject Integration 12
12. Project Ideas & Backend Development Plans 13
13. Research Paper Framework 14
14. Resume-Ready Project Description 15
15. Conclusions & Future Roadmap 16

1. Executive Summary

This report documents a comprehensive technical consultation session in which an engineering student evolved from a simple question about hosting a server using Ngrok to designing and deploying a full 4-node private cloud infrastructure. The session covered system architecture, operating system selection and installation, AI inference deployment, remote access configuration, and academic research paper planning.

The project culminated in the successful installation of Ubuntu Server on a physical machine, deployment of the Ollama AI inference engine running the LLaMA 3 language model, and configuration of SSH-based remote access. This document serves as a complete record of the decisions made, problems encountered, solutions applied, and the technical reasoning behind every architectural choice.

2. Background & Initial Problem Statement

The project originated from a straightforward question: whether a server built on a remote PC using Ngrok would run continuously and be available for free. This question reflected a common misconception about tunneling tools versus true server infrastructure, and served as the entry point for a much deeper technical exploration.

2.1 The Original Question

The initial inquiry was whether Ngrok could serve as a reliable, cost-free, 24/7 hosting solution. The answer required a thorough explanation of how Ngrok fundamentally operates and why it is architecturally unsuited to serve as a production hosting environment.

2.2 Available Resources

A key factor enabling the expanded project vision was the availability of multiple free, unused laboratory computers. This transformed the project scope from a simple hosting question into an opportunity to build a legitimate distributed computing cluster.

Resource	Description
Available Hardware	Multiple unused laboratory PCs
Storage	512 GB SSD (per primary node)
Network	Ethernet-connected LAN with internet access
USB Drive	64 GB USB (used as installation medium)
Existing Skills	Flutter development, Firebase, YOLO object detection, Spring Boot basics

3. Discovery: Why Ngrok Is Not a Production Solution

The session began with a detailed analysis of Ngrok's architecture and limitations. This analysis was critical in establishing the need for a proper infrastructure design.

3.1 How Ngrok Works

Ngrok operates by creating a temporary encrypted tunnel between the public internet and a locally running service on the user's machine. It is a developer tool designed for testing and demonstration purposes, not for production deployments.

3.2 Critical Limitations Identified

Limitation	Impact on 24/7 Hosting
PC Must Stay ON	If the host machine sleeps, shuts down, or loses power, the server becomes completely unreachable.
Session Expiry	Free Ngrok sessions disconnect after approximately 2 hours, requiring manual restart.
Dynamic URL	Every Ngrok restart generates a new public URL, making stable API endpoints impossible.
Bandwidth Restrictions	Free plan imposes strict limits unsuitable for any production application.
No Cluster Support	Ngrok provides one tunnel per service and is not designed for distributed architectures.

The conclusion reached was unambiguous: Ngrok is a demonstration and testing tool, not an infrastructure tool. For genuine 24/7 availability, a proper cloud-hosted or self-hosted solution is required.

4. Project Vision: Building a Mini Data Center

Upon recognising the inadequacy of Ngrok for the stated goals, the project vision expanded dramatically. The availability of multiple laboratory computers presented an opportunity to build a genuine private cloud infrastructure — effectively a homelab cluster capable of hosting real applications, an AI inference engine, a database, and a publicly accessible API.

4.1 High-Level Goals

- Create a server system accessible from anywhere in the world
- Utilise repurposed hardware to minimise costs
- Deploy a local AI model (LLM) without reliance on paid APIs
- Build a production-quality backend service
- Generate a publishable research paper from the project
- Create a compelling resume portfolio entry

4.2 Conceptual Architecture

The core architectural decision was to designate one machine as the public-facing gateway and route all internal traffic through it, keeping other nodes protected behind the local network.

Node	Assigned Role
PC1 (Gateway)	Public entry point — Nginx reverse proxy, SSL termination, DuckDNS domain
PC2 (Backend)	Spring Boot application server — API logic and service orchestration
PC3 (Database)	MySQL or MongoDB — persistent storage and data management
PC4 (AI Server)	Ollama LLM inference engine — YOLO models and ML workloads

5. System Architecture Design

5.1 Network Flow Diagram

The complete traffic flow through the system is as follows:

```
Internet
    |
Router (Port Forwarding: 80, 443)
    |
PC1 — Gateway (Nginx + DuckDNS + Certbot SSL)
    |
    +--------+--------+--------+
    |        |        |        |
   PC2      PC3      PC4
(Backend) (Database) (AI/LLM)
```

5.2 Technology Stack

Layer	Technology Used
Operating System	Ubuntu Server 22.04 LTS (bare metal)
Reverse Proxy / Gateway	Nginx (open-source, free)
SSL / HTTPS	Certbot (Let's Encrypt — free certificates)
Dynamic DNS	DuckDNS (free subdomain)
Containerisation	Docker (for backend services)
Orchestration (planned)	Kubernetes or Docker Swarm
Backend Framework	Spring Boot (Java)
Database	MySQL / MongoDB

AI Inference Engine	Ollama (local LLM runtime)
AI Model	LLaMA 3 / Mistral (via Ollama)
Automation	Shell scripting + cron jobs
Remote Access	SSH + Cloudflare Tunnel

5.3 Networking Strategy

Two access models were defined during the session:

- Local Network Only (Option A): All PCs connected via LAN, accessible using private IP addresses such as 192.168.1.x. Sufficient for internal development.
- Global Internet Access (Option B): Requires either port forwarding on the router, a static public IP or Dynamic DNS, and a reverse proxy to route traffic internally.

A critical constraint specific to the Indian ISP environment was identified: most consumer and institutional ISPs assign dynamic IP addresses and may employ CGNAT, which prevents port forwarding. This was flagged as a potential blocker requiring either a Cloudflare Tunnel workaround or a VPS-based relay.

6. Technical Challenges & Problems Faced

Multiple categories of challenges were anticipated and encountered throughout the project. This section documents each problem alongside its root cause and the resolution applied.

6.1 Infrastructure & Hardware Challenges

Challenge	Status / Resolution
Power Continuity	Lab PCs not designed for 24/7 operation. Resolution: Disable sleep/hibernation in BIOS, consider UPS for backup power.
Overheating Risk	Continuous operation generates heat. Resolution: Ensure adequate ventilation; monitor CPU temperatures.
Network Instability	LAN switch required for stable cluster connectivity. Resolution: Dedicated managed switch recommended.
Hardware Performance	CPU-only inference is significantly slower than GPU-accelerated. Resolution: Use quantised models; implement streaming responses.

6.2 ISP & Networking Challenges

- Dynamic IP addresses: Most Indian ISPs do not provide static IPs. Resolved with DuckDNS auto-update cron job.

- CGNAT: Some ISPs share a single public IP among multiple customers, making port forwarding impossible. Resolved with Cloudflare Tunnel (no port forwarding required).
- Port blocking: ISPs may block ports 80 and 443. Resolved with Cloudflare Tunnel which operates over HTTPS.

6.3 Installation-Specific Problems Encountered

During the actual Ubuntu Server installation, two specific technical problems were encountered and resolved in real time:

Problem 1: Installation Failure at apt Configuration

During the installation process, an error occurred at the 'configuring apt / curtin command in-target' stage. This presented as a terminal error screen halting the installation.

- Root Cause: Temporary network timeout when reaching the Ubuntu mirror server (in.archive.ubuntu.com). Common on institutional networks.
- Resolution Applied: Restarted the installer from the error screen. On the second attempt, when the mirror verification screen appeared, the installer was advanced immediately with 'Continue without waiting' to bypass the timeout check.
- Outcome: Installation completed successfully on the second attempt.

Problem 2: USB Unmount Failure at Reboot

After installation completed, the system displayed a 'Failed unmounting cdrom.mount — Please remove the installation medium' message and would not reboot cleanly.

- Root Cause: The installer could not automatically eject the USB drive.
- Resolution Applied: Physical removal of the USB drive followed by pressing Enter.
- Outcome: System rebooted cleanly into the installed Ubuntu Server environment.

6.4 Security Considerations

Opening any server to the public internet introduces significant security responsibilities. The following minimum security measures were identified as mandatory:

- UFW Firewall: Enable and allow only ports 80 and 443 publicly.
- SSH Key Authentication: Replace password authentication with key-based SSH.
- No Direct Database Exposure: Database nodes must never be directly accessible from the internet.
- Unused Port Closure: All services not in use should have their ports closed by default.

7. Operating System Selection & Rationale

A significant portion of the session was dedicated to evaluating operating system options for the cluster nodes. Several installation approaches were considered.

7.1 Options Evaluated

Option	Assessment
Ubuntu Server — Bare Metal	RECOMMENDED. Maximum performance. No virtualisation overhead. Ideal for 24/7 server operation. Requires wiping existing OS.
Ubuntu Desktop — Bare Metal	Not ideal for servers. GUI consumes significant RAM and CPU. Suitable only if visual interface is mandatory.
Dual Boot	Incompatible with 24/7 server requirements. Cannot run both operating systems simultaneously. Not recommended.
Virtual Machine (VirtualBox/VMware)	Suitable for learning only. Poor performance for 24/7 operation. Networking complications with port forwarding. Not recommended.
Omakub	Clarified not to be an OS. Omakub is a post-installation customisation script for Ubuntu Desktop. Irrelevant for server nodes.

7.2 Version Selected

Ubuntu Server 22.04 LTS was selected as the operating system for all cluster nodes. Key reasons:

- Long-Term Support: Security updates guaranteed until 2027.
- Industry Standard: Widely used in production cloud environments.
- Ecosystem Compatibility: Native support for Docker, Kubernetes, Nginx, Spring Boot, and Ollama.
- Community: Largest community of any Linux server distribution, ensuring ease of troubleshooting.

7.3 OS Customisation Approach

Rather than building a custom kernel, the project defines a 'custom OS layer' on top of Ubuntu Server through:

- Pre-installation of required tools: Docker, Nginx, curl, git
- Automated startup scripts that restore all services on boot
- Role-specific hostname assignment (e.g., jaadu-node1, jaadu-gateway)
- Custom MOTD (Message of the Day) displaying cluster status on login
- Bash aliases for common cluster management commands
- Optional future step: Generating a custom ISO using Cubic for one-click deployment

8. Step-by-Step Installation Chronicle

The following section documents the complete installation procedure as executed, including all decisions made and their rationale.

8.1 Bootable USB Creation

1. Downloaded Ubuntu Server 22.04 LTS ISO from ubuntu.com/download/server.
2. Installed Rufus (rufus.ie) on a Windows machine.
3. Inserted 64 GB USB drive and selected it in Rufus.
4. Selected Ubuntu Server ISO as the boot source.
5. Settings used: Partition scheme = MBR, File system = FAT32, Persistent partition size = 0.
6. When prompted to download updated GRUB bootloader: selected YES.
7. When prompted about write mode: selected ISO mode (Recommended).
8. Process completed in approximately 5-10 minutes. USB ready.

8.2 Ubuntu Server Installation

9. Inserted bootable USB and restarted the target PC.
10. Entered boot menu via F12/F2/DEL and selected USB device.
11. Installation type selected: Ubuntu Server (standard — not minimised).
12. Third-party drivers option: left unchecked.
13. Network configuration: DHCP accepted automatically. Ethernet connection active (IP: 10.2.125.242).
14. Proxy configuration: left blank (no proxy in use).
15. Mirror: in.archive.ubuntu.com (auto-selected, India-optimised). Verified as reachable.
16. Storage: selected 'Use entire disk' with LVM enabled. Encryption: disabled. Entire 512 GB SSD wiped and formatted.
17. Profile setup: Name = ULTRON, Server name = ultron, Username = ultron, Password set.
18. Ubuntu Pro: SKIPPED.
19. OpenSSH: ENABLED (critical for remote access).
20. Featured snaps: ALL UNCHECKED (manual installation preferred for control).
21. Installation ran for approximately 10-15 minutes. First attempt failed at apt configuration (mirror timeout). Second attempt succeeded.
22. On completion: USB removed, system rebooted into installed Ubuntu Server.

8.3 Post-Installation First Boot

After first boot, cloud-init logs filled the screen during the first login. This is normal behaviour on Ubuntu Server's initial startup. Pressing Enter revealed the login prompt.

```
ultron login: ultron
Password: [entered]
ultron@ultron:~$
```

The server was operational and ready for software deployment.

8.4 System Update & Ollama Installation

Commands executed after first login:

```
sudo apt update && sudo apt upgrade -y
ip a   # confirmed IP: 10.2.125.242
curl -fsSL https://ollama.com/install.sh | sh
ollama run llama3
```

Ollama installed successfully, confirmed running as a systemd service on port 11434. LLaMA 3 model was downloaded (approximately 3-5 GB) and launched successfully.

9. AI Infrastructure: Ollama & Local LLM Deployment

9.1 What Was Deployed

Ollama is an open-source local LLM inference engine that allows running large language models entirely on-premises without any API costs or internet dependency after initial download.

Component	Details
Inference Engine	Ollama v1.x
Model Deployed	LLaMA 3 (Meta AI)
API Endpoint	http://localhost:11434
Mode	CPU inference (no GPU)
Service Management	systemd service (auto-start on boot)

9.2 Performance Expectations & Optimisations

Running LLMs on CPU-only hardware results in noticeably slower inference compared to GPU-accelerated alternatives. The following optimisations were identified:

- Use smaller, quantised models: Mistral or Gemma instead of larger LLaMA variants.
- Enable streaming responses: Set 'stream: true' in API calls so the user sees token-by-token output, creating the appearance of faster responses.
- Dedicated AI node: PC4 runs only the AI service, with no competing processes.
- Short, focused prompts: Avoid open-ended requests that generate lengthy completions.

9.3 Enabling Network Access

By default Ollama binds only to localhost. To enable access from other devices on the network, the systemd service file was modified:

```
sudo nano /etc/systemd/system/ollama.service
# Change: ExecStart=/usr/local/bin/ollama serve
# To:    ExecStart=/usr/local/bin/ollama serve --host 0.0.0.0
sudo systemctl daemon-reload && sudo systemctl restart ollama
sudo ufw allow 11434
```

9.4 AI Models & Use Cases Planned

Model / Approach	Intended Use Case
LLaMA 3 / Mistral (via Ollama)	General-purpose chatbot, Q&A system, ChatGPT-like interface
YOLOv8	Object detection API — dashcam, safety monitoring
RAG (Retrieval-Augmented Generation)	Question paper analyser, domain-specific knowledge base
FAISS Vector Database	Semantic search over stored academic documents
Custom Mini-Transformer (planned)	Educational project — build and train a small LLM from scratch using PyTorch

10. Remote Access Setup: SSH & Cloudflare Tunnel

10.1 SSH Configuration

SSH (Secure Shell) was enabled during the Ubuntu Server installation by selecting the OpenSSH Server option. This allows full terminal-based remote control of the server from any device on the same network.

Detail	Value
SSH Status	Active and running (confirmed via systemctl status ssh)
Port	22 (default)
Authentication	Password authentication (key-based recommended for production)
Server IP	10.2.125.242 (local network)

From a Windows laptop, connection is established via:

```
ssh ultron@10.2.125.242
```

This enables copy-paste command entry, file transfer via tools like WinSCP, and full remote management without physical access to the server.

10.2 Remote GUI Access (Optional)

Three options for visual GUI-based remote access were discussed:

- Full Desktop (ubuntu-desktop): Highest resource usage. Not recommended for server nodes.
- Lightweight GUI (XFCE4 + XRDP): Significantly lighter than full desktop. Suitable if visual interface is occasionally needed.

- Cockpit Web Interface: Browser-based system monitoring and management at http://server-ip:9090. Recommended approach.

10.3 Global Internet Access via Cloudflare Tunnel

To make the server accessible from outside the local network (different WiFi, mobile data, etc.), Cloudflare Tunnel was identified as the optimal solution, particularly given the constraints of Indian ISPs.

Comparison Point	Port Forwarding vs. Cloudflare Tunnel
ISP Compatibility	Port forwarding blocked by many ISPs. Cloudflare Tunnel works universally.
CGNAT	Port forwarding incompatible with CGNAT. Cloudflare Tunnel unaffected.
College/Hostel WiFi	Port forwarding not possible. Cloudflare Tunnel works.
Security	Port forwarding exposes server directly. Cloudflare Tunnel proxied and encrypted.
Cost	Both free.
24/7 Operation	Both require PC to remain powered on.

Cloudflare Tunnel can also expose SSH globally:

```
sudo apt install cloudflared -y
cloudflared tunnel --url ssh://localhost:22
```

11. Academic Subject Integration

The project was mapped against the student's academic curriculum to demonstrate how theoretical subjects directly apply to this practical infrastructure. This mapping is valuable both for academic understanding and for articulating project value in interviews.

Academic Subject	Application in This Project
Cloud Computing	Core architecture concept. The 4-PC cluster directly implements private cloud principles: virtualisation, resource pooling, IaaS, load balancing, fault tolerance.
Operating Systems	Ubuntu Server manages all process scheduling, memory allocation, and filesystem operations. Docker containers are OS-level isolated processes.
Shell Scripting	Automation of server startup, DuckDNS IP updates, scheduled tasks via cron jobs. The DevOps 'glue' of the system.
Database Management System (DBMS)	PC3 hosts MySQL/MongoDB. Concepts of indexing, transactions, query optimisation, and normalisation directly improve system performance.

Machine Learning & AI	PC4 hosts YOLO and Ollama. Model deployment, inference pipelines, and data preprocessing are applied in production context.
Computer Organisation & Architecture	CPU and RAM allocation across nodes, memory hierarchy awareness, I/O optimisation for database and storage nodes.
Software Development / SDLC	Agile methodology applied: each PC brought online as an independent sprint. SDLC phases visible: requirements, design, development, testing, deployment.

12. Project Ideas & Backend Development Plans

12.1 Spring Boot Backend Options

Several backend project concepts were evaluated for their technical merit and resume impact:

Backend Project	Resume Impact
AI-Based Detection API (YOLO + Spring Boot + Flutter)	VERY HIGH — combines ML, backend, and cluster architecture
Distributed File Storage System (mini Google Drive)	HIGH — demonstrates distributed systems and DBMS concepts
Microservices Architecture (multi-node services)	HIGH — directly reflects industry cloud patterns
Real-Time Chat Backend (WebSocket + DB)	MEDIUM-HIGH — integrates with existing Flutter chat app
Smart Dashcam AI (YOLO + event detection)	HIGH — unique specialised application

12.2 Quota-Based Cloud Storage System

One specific project concept was developed in detail: a cloud storage system where each user receives a 200 MB storage quota from the 512 GB SSD. Key technical considerations:

- Math validation: 200 users x 200 MB = 40 GB — well within 512 GB capacity.
- File isolation: Each user's files stored under /storage/user_id/ directory.
- Quota enforcement: Backend checks used_space + file_size <= 200MB before accepting uploads.
- Metadata storage: User info, file metadata, and storage usage tracked in database.
- Advanced options: Distributed storage across nodes, file replication, chunked upload for large files.

12.3 RAG-Based Question Paper Analyser

A highly practical project concept emerged from a prior personal experience: uploading question papers to an AI system that extracts important topics and enables filtered question retrieval. This was formalised as a RAG (Retrieval-Augmented Generation) pipeline:

- Input: PDF question papers
- Processing: Text extraction, chunking, embedding generation (FAISS vector database)
- Topic extraction: Ollama LLM prompted to identify key academic topics
- User interface: Filter buttons per topic, semantic search returning relevant questions
- Tech stack: Python (RAG pipeline), FAISS, Ollama/Mistral, Spring Boot, Flutter

13. Research Paper Framework

13.1 Paper Title

"Design and Implementation of a Distributed Edge Computing Platform using a Custom Linux-Based Cluster OS with Integrated AI Inference Services"

13.2 Core Contributions

23. Design of a low-cost distributed edge infrastructure using repurposed hardware.
24. Development of a custom lightweight server OS configuration for cluster deployment.
25. Integration of microservices-based backend with distributed compute nodes.
26. Incorporation of AI inference as a modular, independently scalable service.
27. Evaluation of system performance in resource-constrained environments.

13.3 Paper Structure

Section	Content Summary
Abstract	Cost-effective distributed edge platform using repurposed hardware, custom server OS, Spring Boot microservices, and optional AI inference.
Introduction	Problem: high cloud costs and vendor lock-in. Solution: self-hosted low-cost cluster. Relevance to educational institutions.
Related Work	Cloud computing, edge computing, containerisation, RAG and local LLM deployment. Identifies gap in low-cost educational infrastructure.
System Architecture	4-node cluster design, role assignment, traffic flow, technology stack.
Methodology	OS configuration, cluster formation, backend design, AI integration, networking.
Implementation	Ubuntu Server, Spring Boot, Docker, Nginx, MySQL, Ollama — specific commands and configuration.

Evaluation	API latency, AI inference time, CPU/RAM utilisation, scalability testing, fault tolerance.
Conclusion & Future Work	Achievements, limitations, and future enhancements (Kubernetes, GPU, CI/CD).

13.4 Evaluation Metrics

- API response latency under varying loads
- AI inference time on CPU-only hardware
- CPU and memory utilisation per node
- Scalability: performance change when adding nodes
- Fault tolerance: service continuity when one node fails

13.5 Connection to Fog Computing

The system aligns precisely with the definition of fog computing: computation performed at the network edge, physically close to data sources, rather than in a remote centralised cloud. This positioning strengthens the academic contribution. Recommended terminology for viva and paper:

"Fog computing enables low-latency distributed processing by bringing computation closer to the data source, which is exactly what our cluster-based system achieves."

14. Resume-Ready Project Description

14.1 Project Title

Mini Private Cloud / Homelab Cluster — 4-Node Distributed System

14.2 Bullet Points

- Designed and deployed a 4-node private cloud infrastructure using repurposed laboratory PCs, simulating a real-world cloud environment.
- Configured a gateway server with Nginx reverse proxy to route external traffic securely across internal services.
- Enabled global access using Dynamic DNS (DuckDNS) and HTTPS (Certbot / Let's Encrypt) with router-level port forwarding.
- Containerised backend services using Docker, ensuring isolated and scalable deployments.
- Architected system with dedicated nodes for backend services, database management, and AI workloads.
- Implemented multi-tier architecture: frontend → gateway → backend → database → AI server.
- Automated server tasks using shell scripting and cron jobs (startup scripts, IP updates).
- Applied distributed systems and cloud computing concepts: resource pooling, load distribution, fault isolation.

- Optimised performance by allocating workloads based on CPU, memory, and I/O characteristics.
- Followed Agile development methodology, incrementally building and testing each node in iterative sprints.

14.3 Technology Stack

Ubuntu Server, Docker, Nginx, DuckDNS, Certbot, MySQL/MongoDB, Ollama, LLaMA 3, Linux Networking, Shell Scripting, Spring Boot

14.4 Recommended Interview Statement

"I built a mini cloud infrastructure using 4 physical machines to understand how distributed systems work in practice. The system handles real HTTP traffic, runs a local AI model, and is accessible from anywhere in the world."

15. Conclusions & Future Roadmap

15.1 Achievements

By the end of the documented session, the following had been accomplished:

- Ubuntu Server 22.04 LTS successfully installed on bare metal hardware.
- Ollama AI inference engine installed and operational.
- LLaMA 3 language model deployed and responding to queries locally.
- SSH remote access configured and verified.
- Complete system architecture designed for all 4 nodes.
- Research paper framework drafted and ready for development.
- Resume-ready project description completed.

15.2 Immediate Next Steps

28. Configure Cloudflare Tunnel for global internet access to the server.
29. Install Nginx on PC1 as the gateway reverse proxy.
30. Configure DuckDNS domain and auto-update cron job.
31. Install Certbot and obtain free SSL certificate.
32. Set up PC2 with Docker and deploy Spring Boot backend.
33. Set up PC3 with MySQL and configure remote access.

15.3 Medium-Term Roadmap

34. Implement Kubernetes cluster for production-grade orchestration.
35. Deploy RAG-based question paper analyser.

36. Build quota-based cloud storage system.
37. Integrate Flutter application with all backend services.
38. Develop and submit research paper to IEEE/Springer venue.

15.4 Long-Term Vision

The project aims to evolve into a fully functional private cloud platform capable of hosting student projects, serving as a college-level infrastructure demonstrator, and providing a published research contribution to the distributed systems and edge computing field. The combination of hands-on hardware experience, modern DevOps tooling, and integrated AI services positions this project as a comprehensive showcase of practical engineering competency.

End of Report — All information sourced from the original technical consultation session.